

Unidad N° 1: Introducción a la Programación Orientada a Objetos

Programación Orientada a Objetos

- *Definición:* **Paradigma de la programación** (una manera de programar específica) donde se **organiza el código en unidades denominadas clases**, de las cuales se crean objetos que se relacionan entre sí para conseguir los objetivos de las aplicaciones.
- Promueve una forma de programar más cercana a como expresaríamos las cosas en la vida real, que permite diseñar mejor las aplicaciones, llegando a mayores cotas de complejidad, sin que el código se vuelva inmanejable.

¿Y la programación estructurada?

- Ventajas:
 - Ofrece una lógica clara y lineal para resolver problemas
 - Implementación rápida: precisa poco diseño previo
 - Es sencilla, dado que no existen conceptos complejos a aprender
 - Favorece la legibilidad en programas cortos
- Casos de uso ideales:
 - Programas de tamaño pequeño (e incluso de tamaño medio)
 - Scripts de administración de sistemas
 - Algoritmos de cálculos matemáticos o lógicos
 - Tareas automatizadas (lotes, batch, cron jobs)
 - Sistemas de periféricos o bajo nivel (microcontroladores)

Desventajas de la programación estructurada

- ***Dificulta el mantenimiento***: complica la inspección de datos
- ***Dificulta la reutilización de código***: poca abstracción de funciones y modularización
- ***Acoplamiento fuerte entre funciones y datos***: lo cual puede crear escenarios inesperados.
- ***Dificultad para escalar una aplicación***: agregar nuevas funcionalidades a la aplicación no ocurre de forma intuitiva
- ***No representa el mundo real***
- ***Dificulta el trabajo en equipo***: se concentra la lógica en pocos archivos y editarlos al mismo tiempo puede ser caótico

Soluciones de la POO

- *Dificulta el mantenimiento* → Encapsulamiento
- *Dificulta la reutilización de código* → Herencia, composición y modularidad
- *Acoplamiento fuerte entre funciones y datos* → Modificadores de acceso
- *Dificultad para escalar una aplicación* → Polimorfismo, SRP
- *No representa el mundo real* → Abstracción, clases y objetos
- *Dificulta el trabajo en equipo* → Modularidad

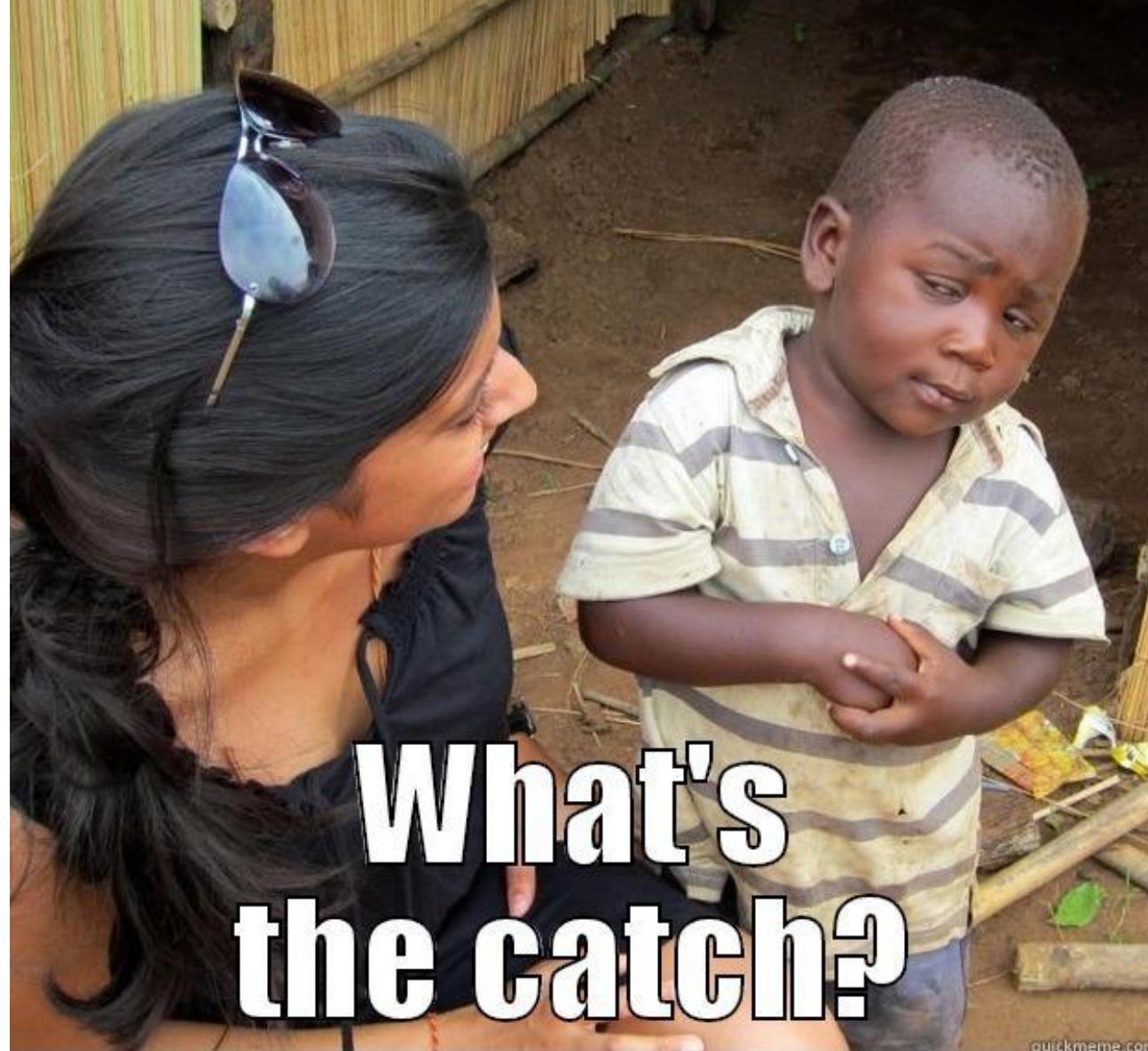
Ventajas de la POO

- Incrementa la mantenibilidad de las aplicaciones
- Facilita la reutilización de código
- Reduce el costo de trabajo en equipo
- Permite escalar aplicaciones de forma más fácil
- Facilita la implementación de testing automatizado
- Facilita la construcción de software complejo

Casos de uso ideales

- Aplicaciones de gran tamaño o de dominio complejo
- Software “extensible” (extensiones, plugins, componentes)
- Sistemas con alta tasa de mantenimiento y/o modificación
- Aplicaciones con interfaces gráficas
- Desarrollo web moderno
- Aplicaciones basadas en objetos de la vida real

That sounds pretty simple



**What's
the catch?**

Desventajas de la POO

- ***Alta complejidad inicial***: tiene una alta curva de aprendizaje
- ***Sobrecarga de diseño***: no es flexible a la hora de diseñar un programa
- ***Mayor consumo de recursos***: posibilidad si no se usan todos los elementos diseñados
- ***Riesgo de mal diseño***: abuso de herencia, posibilidad de código spaghetti, mezcla de estructura con POO, etc.
- ***Mayor tiempo de desarrollo al iniciar un proyecto***
- ***No apto para ciertos dominios o requerimientos***: dominio matemático/lógico, sistemas de tamaño discreto, scripts, etc.

Tip para la vida profesional

- “Nunca se «casen» con un paradigma, tecnología, herramienta, moda de programación”
- No hay herramienta infalible o todo terreno
- **Importante:**
 - Investigar las herramientas
 - Probarlas para experimentar las ventajas/desventajas
 - Definir cuales son las métricas importantes de cada proyecto antes de favorecer una tecnología: dominio del problema, tamaño del sistemas, colaboración en equipo, recursos necesarios, elementos externos (bases de datos, API's), etc.

Diferencias en el código

El sistema de un banco permite transferir montos definidos entre dos cuentas de usuario, una cuenta origen y otra de destino. Para ello siempre es necesario saber los usuarios que intervienen en el proceso y sus respectivas cuentas de origen y destino. El código debe validar que los datos sean correctos y luego realizar la transferencia que impacta en la Base de datos.

```
$campos = ['from_user_id', 'from_account_id', 'to_user_id', 'to_account_id',  
'amount'];  
foreach ($campos as $campo) {  
    if (!isset($_POST[$campo]) || !is_numeric($_POST[$campo])) {  
        exit("Error: campo $key inválido.");  
    }  
}  
  
if ($_POST['amount'] <= 0) {  
    exit("Error: el monto debe ser mayor a 0.");  
}  
  
$db = new PDO('mysql:host=localhost;dbname=test', 'root', '');  
$stmt1 = $db->prepare("UPDATE cuentas SET saldo = saldo - ? WHERE id = ?");  
$stmt1->execute([$_POST['amount'], $_POST['from_account_id']]);  
// Segunda consulta  
$db->commit();  
echo "Transferencia realizada con éxito.";
```

```
public function transferir() {  
    $val = new Validacion($_POST);  
    if (!$val->validarTransferencia()) {  
        exit("Error: " . $val->getError());  
    }  
    $db = new PDO('mysql:host=localhost;dbname=test', 'root', '');  
    $monto = $_POST['amount'];  
    $fromUser = new Usuario($_POST['from_user_id']);  
    $fromAccount = $fromUser->getCuenta($_POST['from_account_id']);  
    // Segundo usuario y cuenta  
    $fromAccount->retirar($monto);  
    $toAccount->depositar($monto);  
    echo "Transferencia realizada con éxito."  
}
```

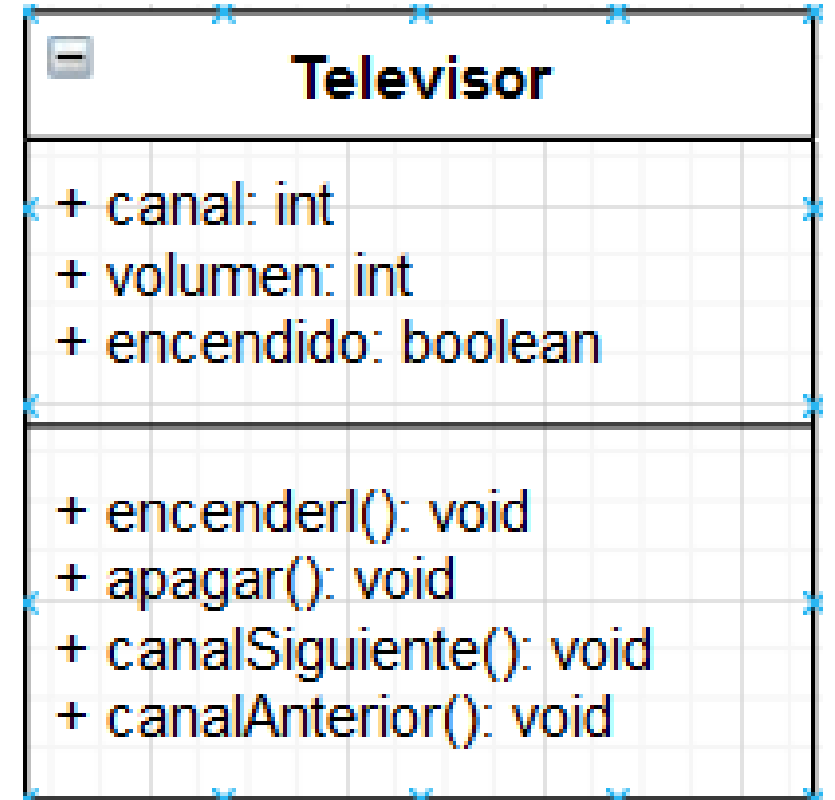
Fundamentos de la Programación Orientada a Objetos

Conceptos básicos

- Clase
- Atributo
- Método
- Objeto
- Estado
- Mensaje

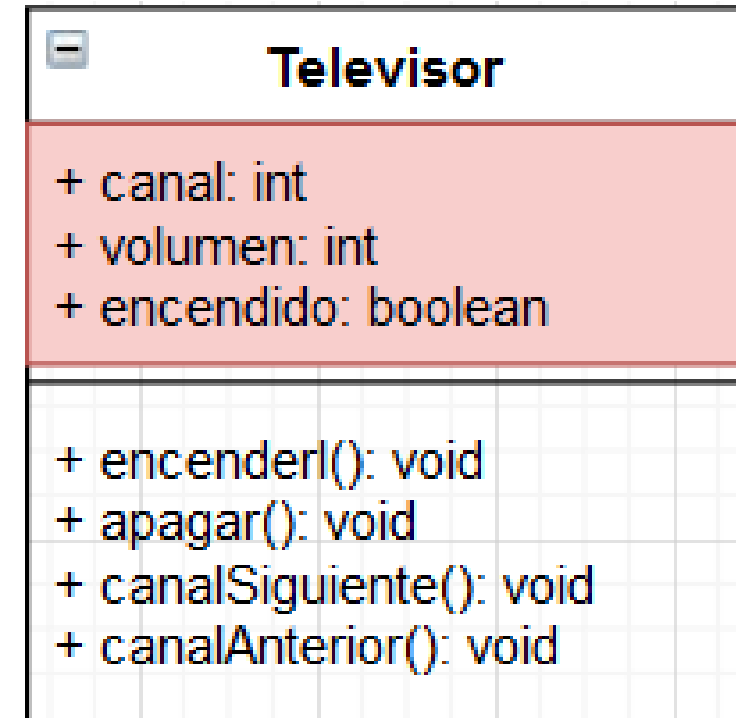
Clase

- Representación de un objeto o entidad de la vida real, compuesta por las características específicas de dicho objeto/entidad (*atributos*) y por las operaciones que definen el comportamiento de la clase (*métodos*).
- Generalización o abstracción de un tipo de objetos en específico.



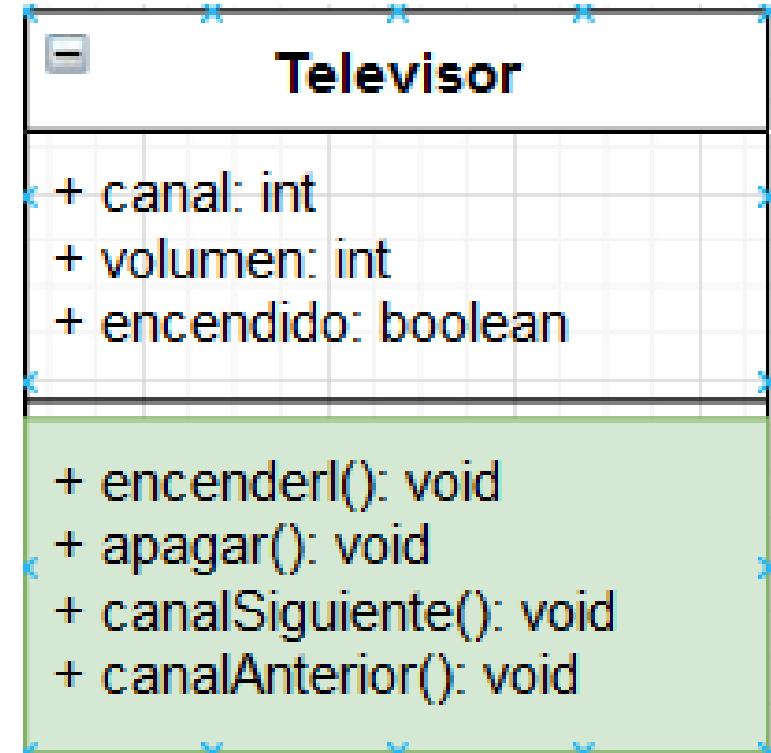
Atributo

- Un atributo es una característica individual que diferencia a un objeto de otro y determinan la estructura de la clase.
- Dentro de la clase, los atributos se definen como variables individuales que poseen un tipo de dato para cada una de ellas.



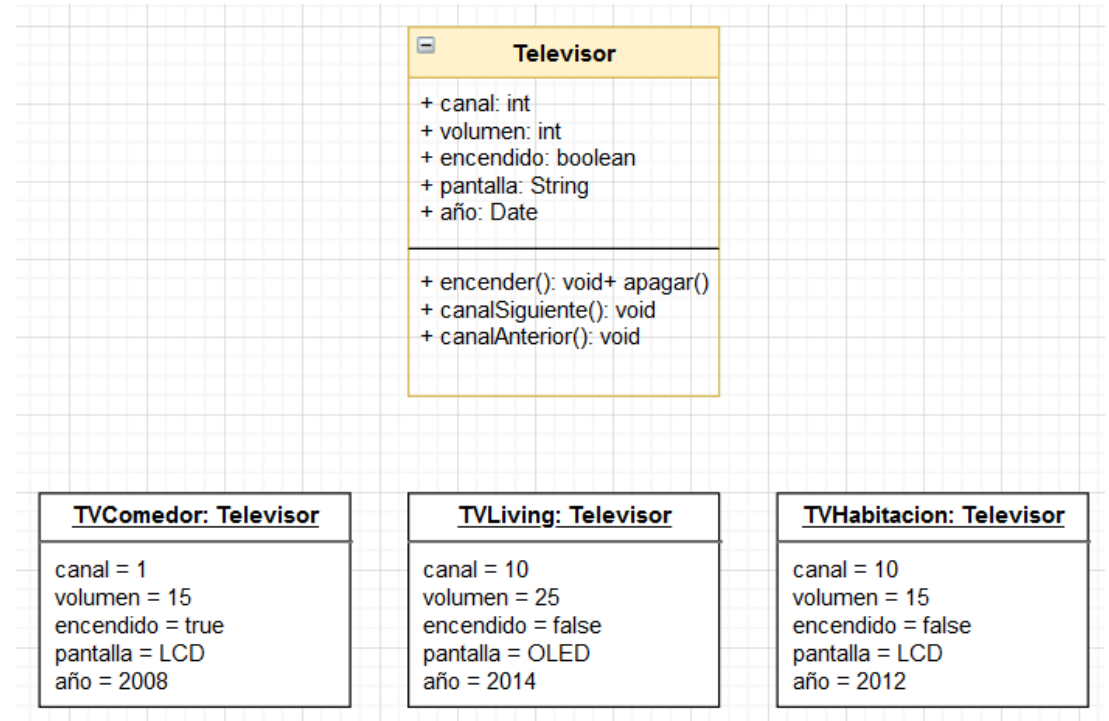
Método

- Un método es un conjunto de instrucciones que realizan una determinada tarea, y por lo general se ejecuta tras la recepción de un mensaje.
- Dentro de la clase, los métodos se definen como funciones que poseen un tipo de dato de retorno y una visibilidad explícita.

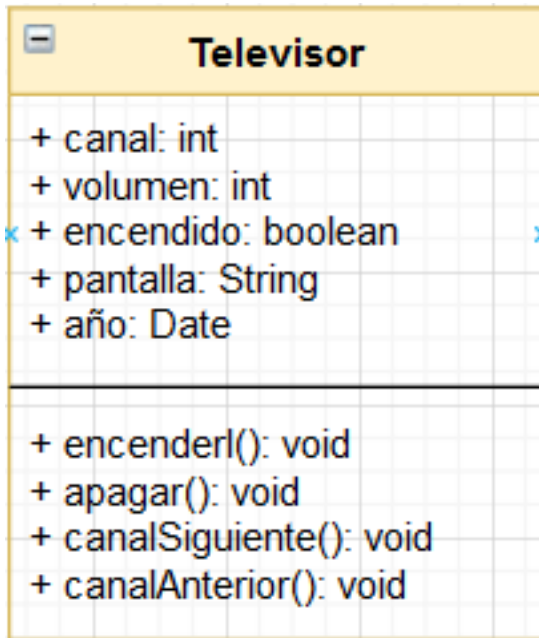


Objeto

- Es una instancia de la clase; es una entidad creada a partir de una clase y posee su propio conjunto de datos y su conjunto de operaciones.
- Al proceso de creación de un objeto se lo denomina ***instanciación***.

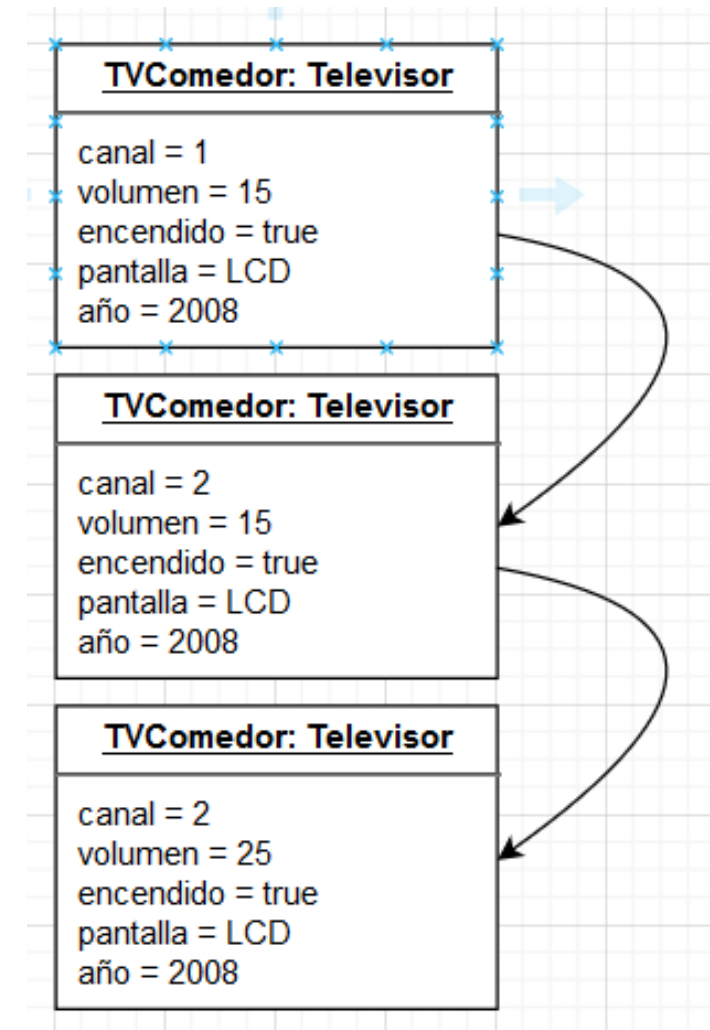


Ejemplo de objetos de una misma clase



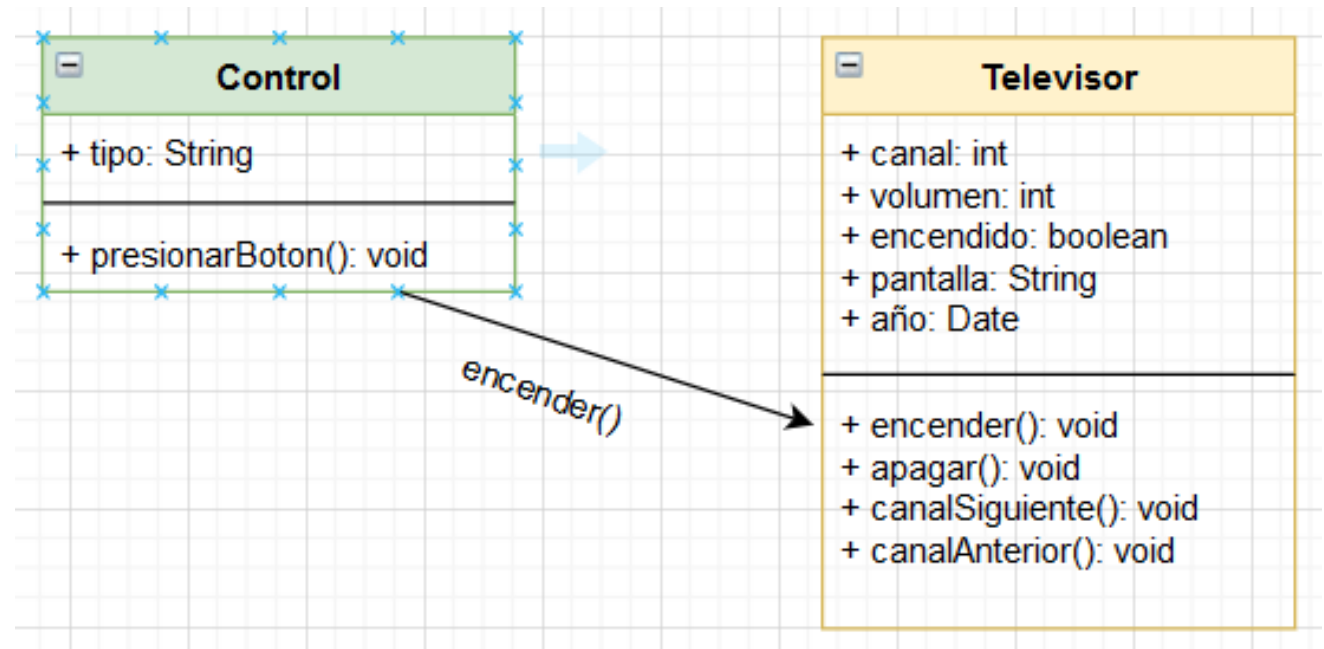
Estado

- Es el conjunto de valores de los atributos de un objeto en un determinado tiempo.
- Cuando cambia el valor de un atributo, cambia el estado de dicho objeto.



Mensaje

- Es una orden dirigida desde un objeto hacia un segundo objeto para invocar la ejecución de un método de este último.
- Esto se conoce como «enviar un mensaje».



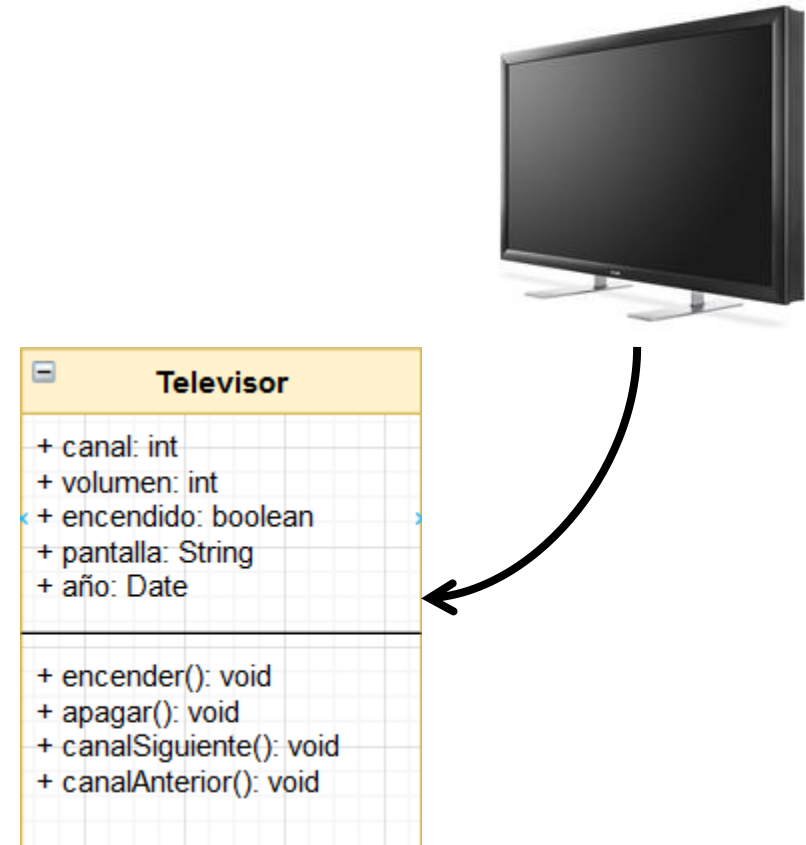
Métodos especiales

- **Constructor:** se ejecuta automáticamente al crear un objeto; inicializa los valores de los atributos. Se define con el mismo nombre de la clase.
- **Getter:** permite acceder al valor de un atributo; se define uno por cada atributo necesario empezando con la palabra “*get*” + el nombre del atributo (en el caso de los booleanos, empieza por “*is*”).
- **Setter:** permite modificar el valor de un atributo; se define uno por cada atributo necesario empezando con la palabra “*set*” + el nombre del atributo.
- **Destructor:** se ejecuta automáticamente cuando un objeto es eliminado; permite realizar tareas de mantenimiento. No es tan usual (en PHP no es necesario implementarlo).

Características fundamentales

Abstracción

- Proceso que permite seleccionar las características relevantes de una clase según el sistema en el cual se verá representada.
- Las clases y objetos de un sistema son entidades abstractas que puede comunicarse con otras entidades sin revelar su funcionamiento interno.
- El objetivo es revelar solamente la información que es relevante y ocultar la complejidad



Encapsulamiento

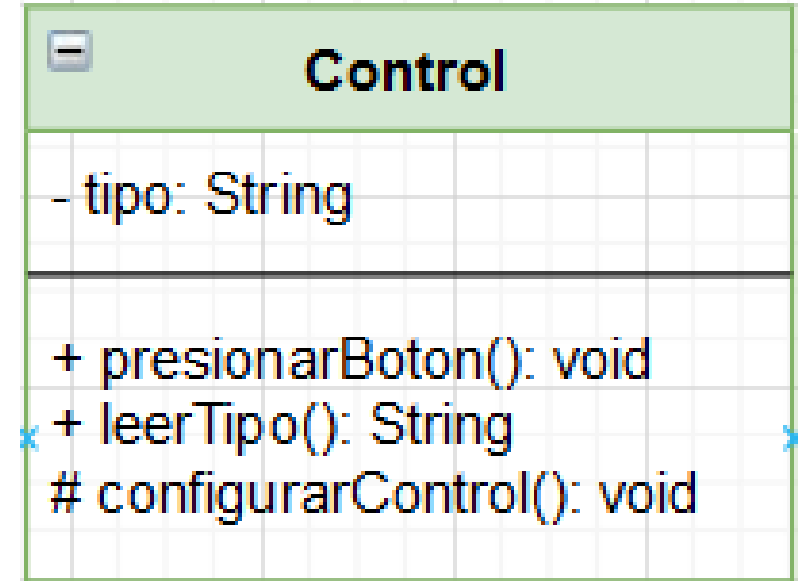
- Capacidad de agrupar datos y operaciones relacionadas bajo la misma unidad de programación (clase) y limitar el acceso de clases externas a la implementación de su estructura; esto último se realiza mediante los modificadores de visibilidad.
- El encapsulamiento define qué mensajes pueden ser enviados a una clase en particular; permite ocultar los detalles internos de un objeto y controlar quién puede acceder a su información.

Televisor	
- canal: int	
- volumen: int	
- encendido: boolean	
- pantalla: String	
- año: Date	
+ encender(): void	
+ apagar(): void	
+ canalSiguiente(): void	
+ canalAnterior(): void	
- cambiarCanal(): void	
- obtenerNombrePantalla(): void	

Modificadores de visibilidad

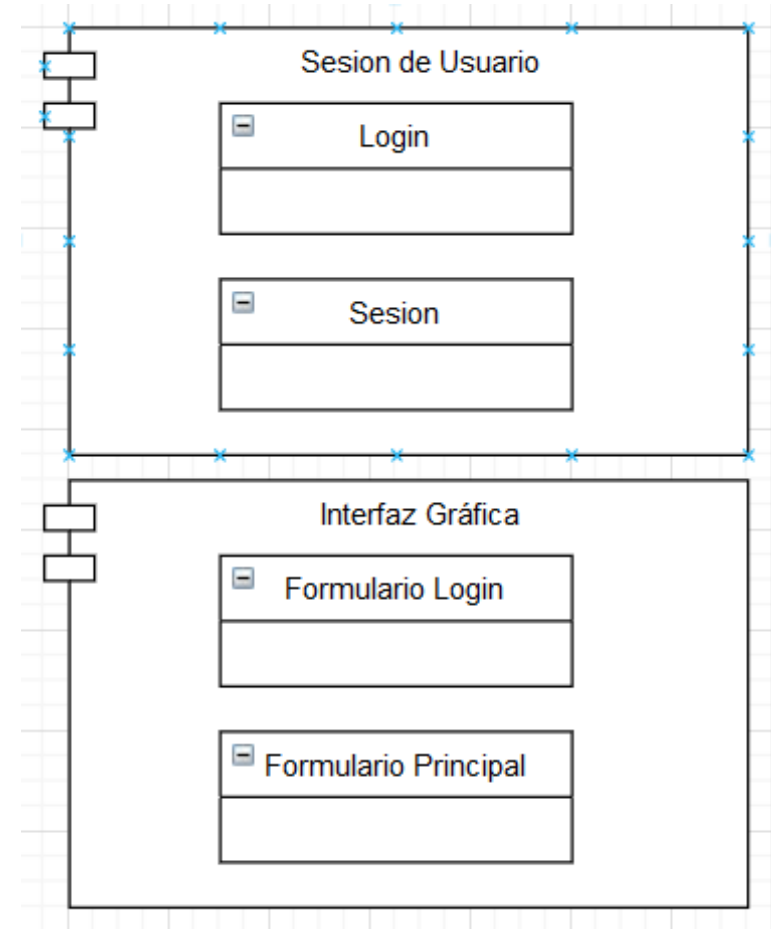
Los modificadores de visibilidad especifican las reglas de acceso a un atributo o un método de clase.

- **public**: define que el elemento puede ser accedido desde cualquier clase.
- **private**: define que el elemento puede ser accedido solamente desde la clase que lo define.
- **protected**: define que el elemento puede ser accedido desde la clase que la define o sus sub-clases.



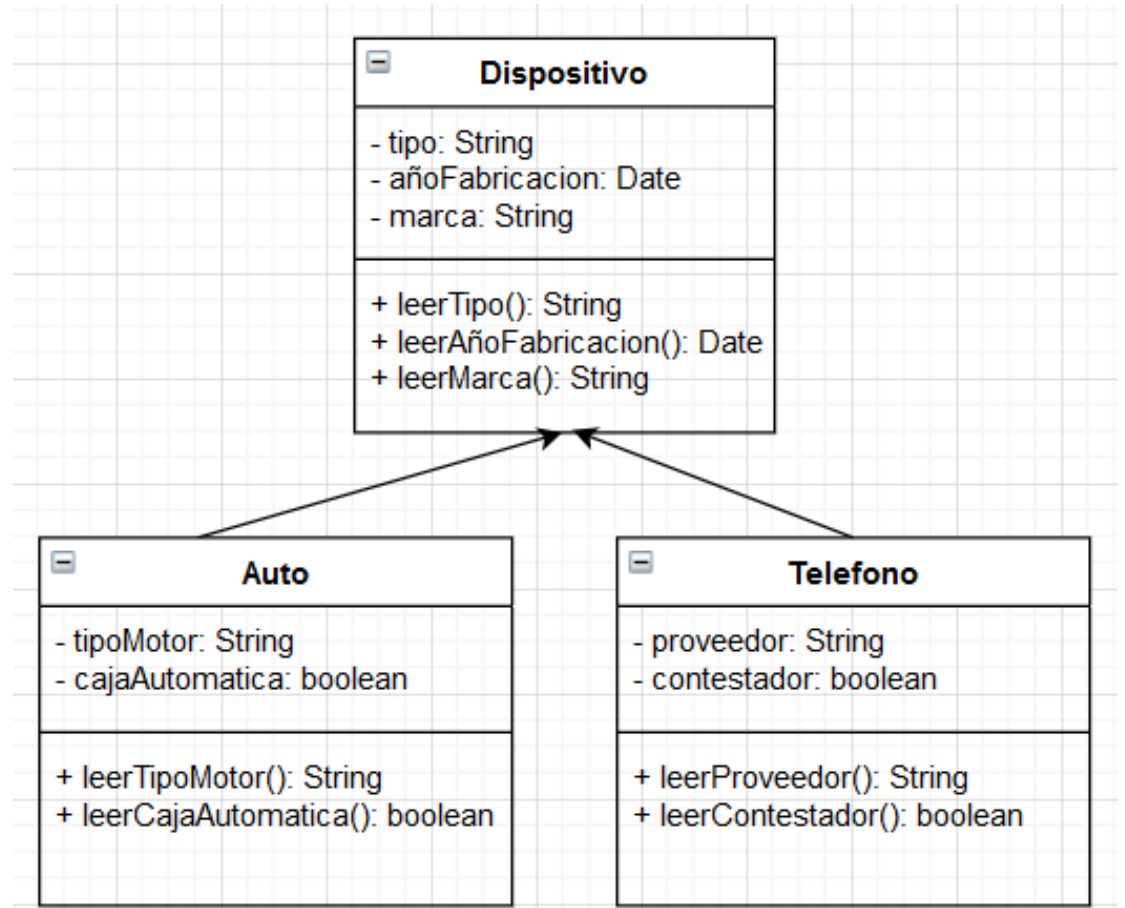
Modularidad

- Propiedad que permite dividir una aplicación o sistema en partes más sencillas llamadas módulos, los cuales permiten agrupar clases que posean un mismo propósito.
- Los módulos permiten reducir la complejidad del código y definir responsabilidades únicas (*Principio de Responsabilidad Simple*).



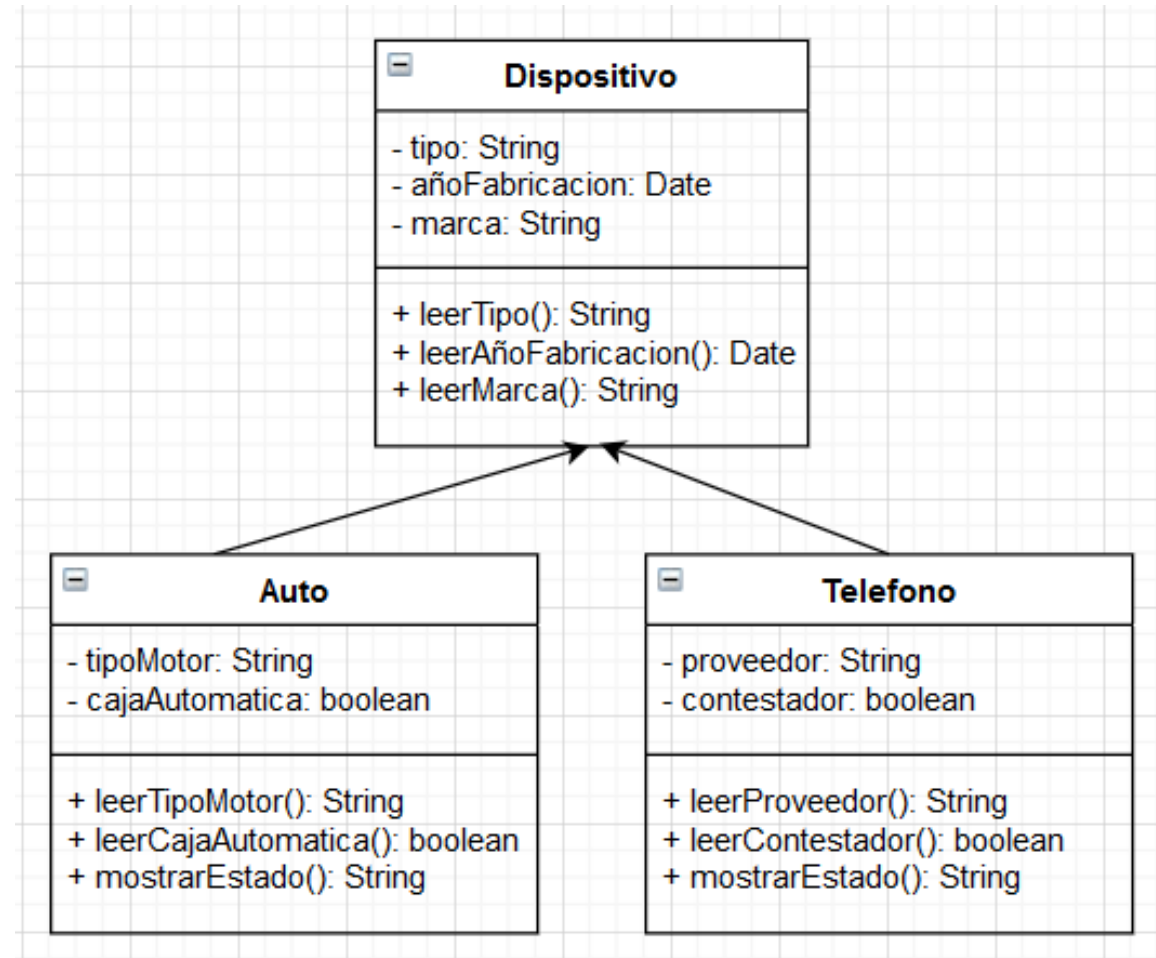
Herencia

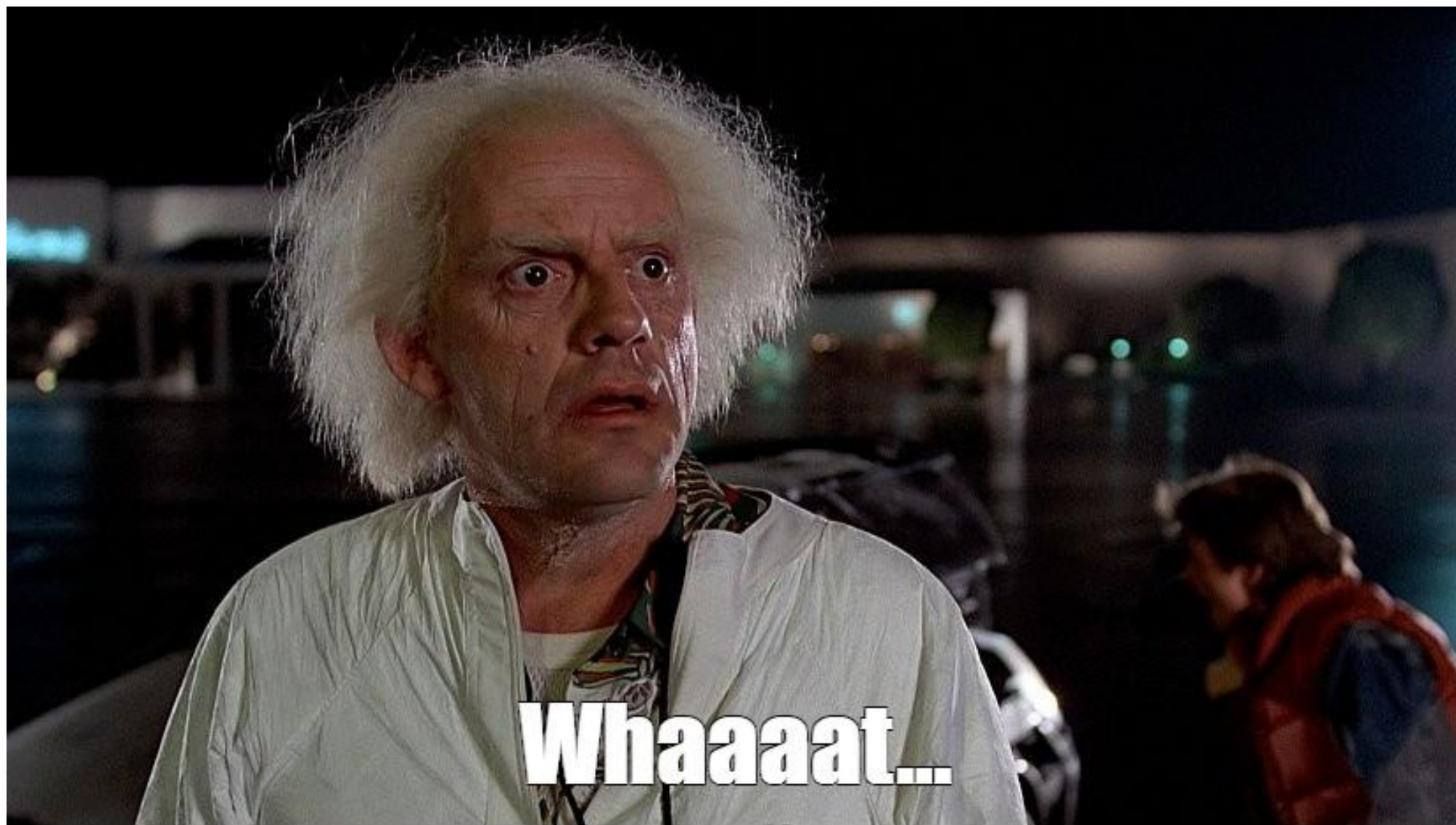
- Es un mecanismo que permite definir clases en base a otras ya existentes, de modo que la subclase obtiene los atributos y métodos de la superclase. De esta forma se va formando un árbol de jerarquía.
- *Subclase* → *Clase hija*.
- *Superclase* → *Clase padre*.



Polimorfismo

- Mecanismo que permite que un método responda de distintas maneras a un mensaje. Esto permite que diversas clases definan el mismo método pero que este opere de manera distinta según su contexto.
- «*Poli*» → *Muchos*.
- «*Morfos*» → *Forma*.



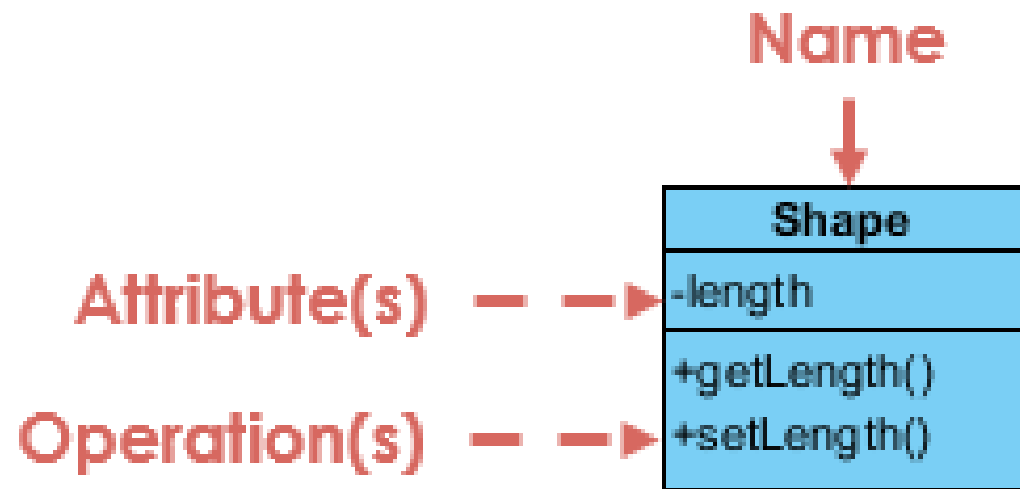


Diagramas de clases UML

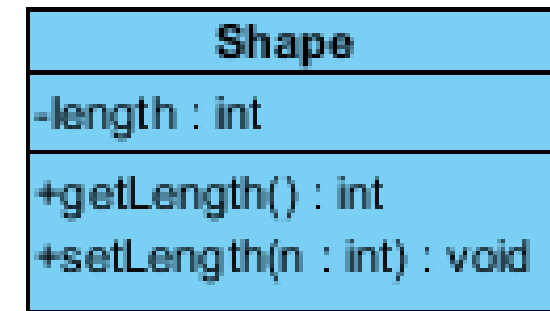
Conceptos

- Un **diagrama de clases** es un tipo de **diagrama estático** que forma parte del lenguaje de modelado UML (**Unified Modeling Language**).
- ¿Para qué se usa?
 - Para diseñar sistemas orientados a objetos previos a su codificación
 - Para documentar los elementos de sistemas ya existentes
 - Para comunicar la estructura de los sistemas entre analistas de sistemas, programadores y diseñadores.
 - Para detectar errores en la estructura de los sistemas antes de su implementación

Ejemplo de un diagrama de clase



Class without signature



Class **with** signature

¿Cómo hacemos un diagrama de clase?

1. Definir el propósito y el alcance del problema
2. Identificar las clases
3. Definir los atributos
4. Definir los métodos
5. Mostrar las relaciones
6. Confeccionar el diagrama UML

Caso de ejemplo

Se nos pide el diseño de un sistema para la liga de básquet nacional. La misma está conformada por distintos equipos, los cuales se identifican mediante un nombre único, la ciudad a la cual pertenecen y la división (dado que pueden descender o ascender). Cada equipo contrata a su equipo de entrenadores, y cada uno de ellos tiene un nombre y apellido, un rol dentro del equipo y la cantidad de años de experiencia. Cada entrenador es responsable del entrenamiento del equipo y la confección de estrategias para los juegos. Cada equipo también cuenta con jugadores, de los cuales interesa registrar el nombre y apellido, la posición en la que juega y el número de camiseta. A su vez, los jugadores pueden entrenar o jugar partidos con sus equipos.

1) Definir el propósito y alcance

- **Propósito:** creación del sistema para la liga nacional de básquet.
- **Alcance:**
 - Elementos y entidades de la LNB
 - Quedan descartados todos los elementos y entidades fuera de este minimundo
 - Quedan descartados todos los elementos internos que sean irrelevantes: instalaciones del equipo, personal administrativo de los equipos, datos personales de los jugadores o entrenadores

2) Identificar las clases

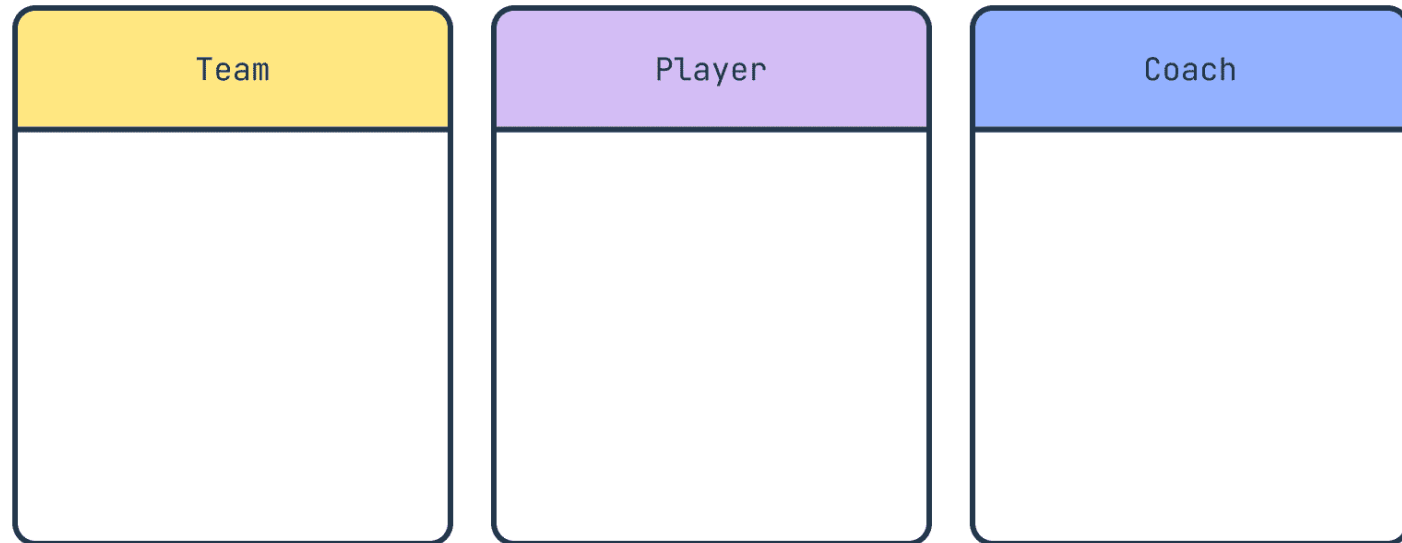
Se nos pide el diseño de un sistema para la liga de básquet nacional. La misma está conformada por distintos equipos, los cuales se identifican mediante un nombre único, la ciudad a la cual pertenecen y la división (dado que pueden descender o ascender). Cada equipo contrata a su equipo de entrenadores, y cada uno de ellos tiene un nombre y apellido, un rol dentro del equipo y la cantidad de años de experiencia. Cada entrenador es responsable del entrenamiento del equipo y la confección de estrategias para los juegos. Cada equipo también cuenta con jugadores, de los cuales interesa registrar el nombre y apellido, la posición en la que juega y el número de camiseta. A su vez, los jugadores pueden entrenar o jugar partidos con sus equipos.

2) Identificar las clases

Se nos pide el diseño de un sistema para la liga de básquet nacional. La misma está conformada por distintos **equipos**, los cuales se identifican mediante un nombre único, la ciudad a la cual pertenecen y la división (dado que pueden descender o ascender). Cada equipo contrata a su equipo de **entrenadores**, y cada uno de ellos tiene un nombre y apellido, un rol dentro del equipo y la cantidad de años de experiencia. Cada entrenador es responsable del entrenamiento del equipo y la confección de estrategias para los juegos. Cada equipo también cuenta con **jugadores**, de los cuales interesa registrar el nombre y apellido, la posición en la que juega y el número de camiseta. A su vez, los jugadores pueden entrenar o jugar partidos con sus equipos.

2) Identificar las clases

- **Equipo** → Representa a un equipo perteneciente a la liga.
- **Jugador** → Representa los jugadores de cada equipo.
- **Entrenador** → Representa a cada miembro del cuerpo técnico de un equipo puntual.



3) Definir los atributos

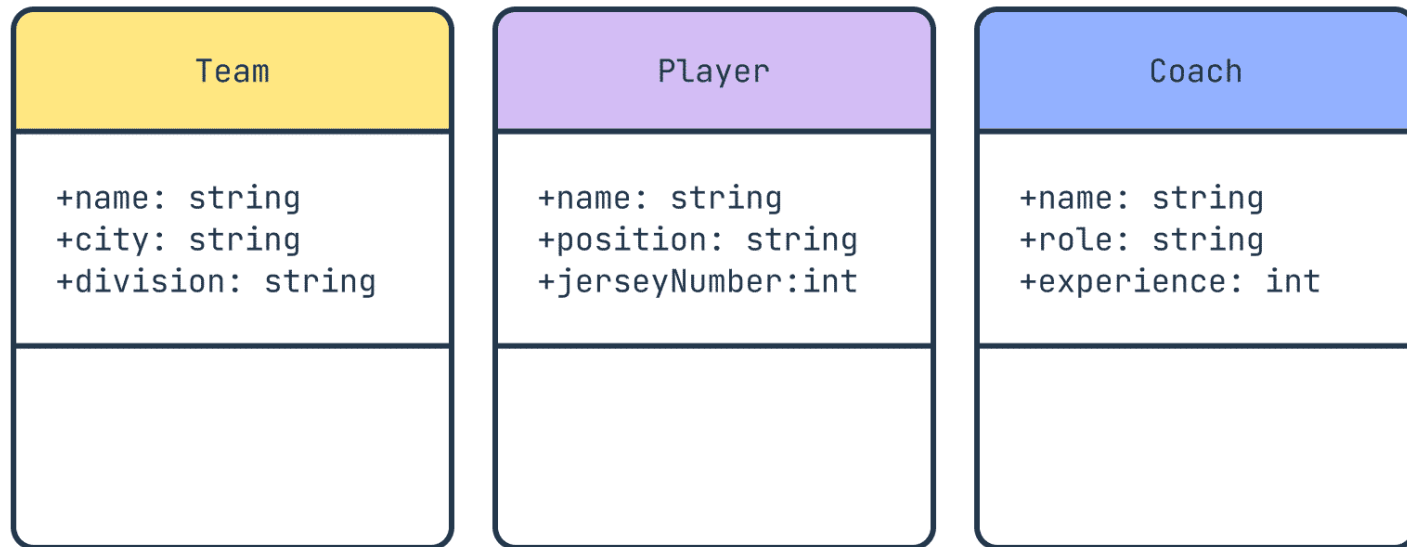
La misma está conformada por distintos **equipos**, los cuales se identifican mediante un nombre único, la ciudad a la cual pertenecen y la división (dado que pueden descender o ascender). Cada equipo contrata a su equipo de **entrenadores**, y cada uno de ellos tiene un nombre y apellido, un rol dentro del equipo y la cantidad de años de experiencia. Cada entrenador es responsable del entrenamiento del equipo y la confección de estrategias para los juegos. Cada equipo también cuenta con **jugadores**, de los cuales interesa registrar el nombre y apellido, la posición en la que juega y el número de camiseta. A su vez, los jugadores pueden entrenar o jugar partidos con sus equipos.

3) Definir los atributos

La misma está conformada por distintos **equipos**, los cuales se identifican mediante un **nombre** único, la **ciudad** a la cual pertenecen y la **división** (dado que pueden descender o ascender). Cada equipo contrata a su equipo de **entrenadores**, y cada uno de ellos tiene un **nombre y apellido**, un **rol** dentro del equipo y la cantidad de **años de experiencia**. Cada entrenador es responsable del entrenamiento del equipo y la confección de estrategias para los juegos. Cada equipo también cuenta con **jugadores**, de los cuales interesa registrar el **nombre y apellido**, la **posición** en la que juega y el **número de camiseta**. A su vez, los jugadores pueden entrenar o jugar partidos con sus equipos.

3) Definir los atributos

- **Equipo** → nombre, ciudad, división.
- **Jugador** → nombre y apellido, posición, número de camiseta.
- **Entrenador** → nombre y apellido, rol, experiencia.



4) Definir los métodos

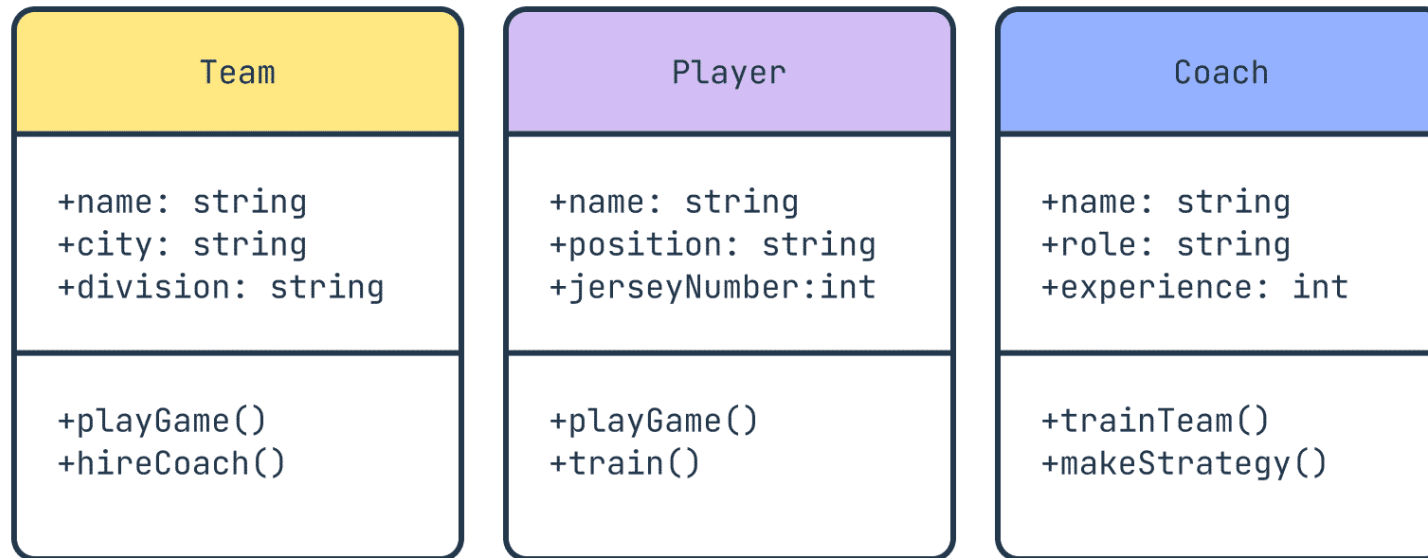
La misma está conformada por distintos **equipos**, los cuales se identifican mediante un nombre único, la ciudad a la cual pertenecen y la división (dado que pueden descender o ascender). Cada equipo contrata a su equipo de **entrenadores**, y cada uno de ellos tiene un nombre y apellido, un rol dentro del equipo y la cantidad de años de experiencia. Cada entrenador es responsable del entrenamiento del equipo y la confección de estrategias para los juegos. Cada equipo también cuenta con **jugadores**, de los cuales interesa registrar el nombre y apellido, la posición en la que juega y el número de camiseta. A su vez, los jugadores pueden entrenar o jugar partidos con sus equipos.

4) Definir los métodos

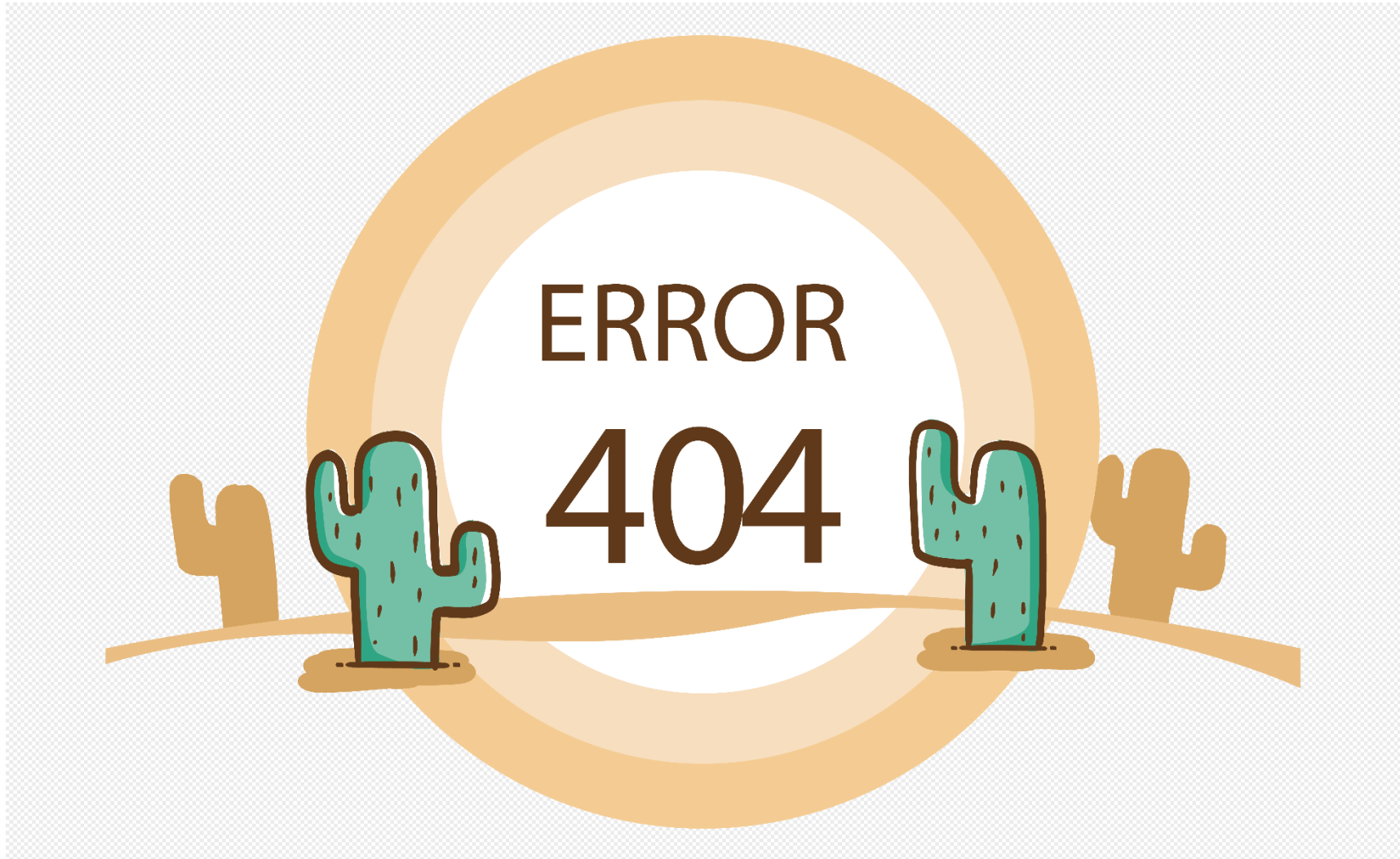
La misma está conformada por distintos **equipos**, los cuales se identifican mediante un nombre único, la ciudad a la cual pertenecen y la división (dado que pueden descender o ascender). Cada equipo **contrata a su equipo** de **entrenadores**, y cada uno de ellos tiene un nombre y apellido, un rol dentro del equipo y la cantidad de años de experiencia. Cada entrenador es responsable del **entrenamiento del equipo** y la **confección de estrategias** para los juegos. Cada equipo también cuenta con **jugadores**, de los cuales interesa registrar el nombre y apellido, la posición en la que juega y el número de camiseta. A su vez, los jugadores pueden **entrenar** o **jugar partidos con sus equipos**.

4) Definir los métodos

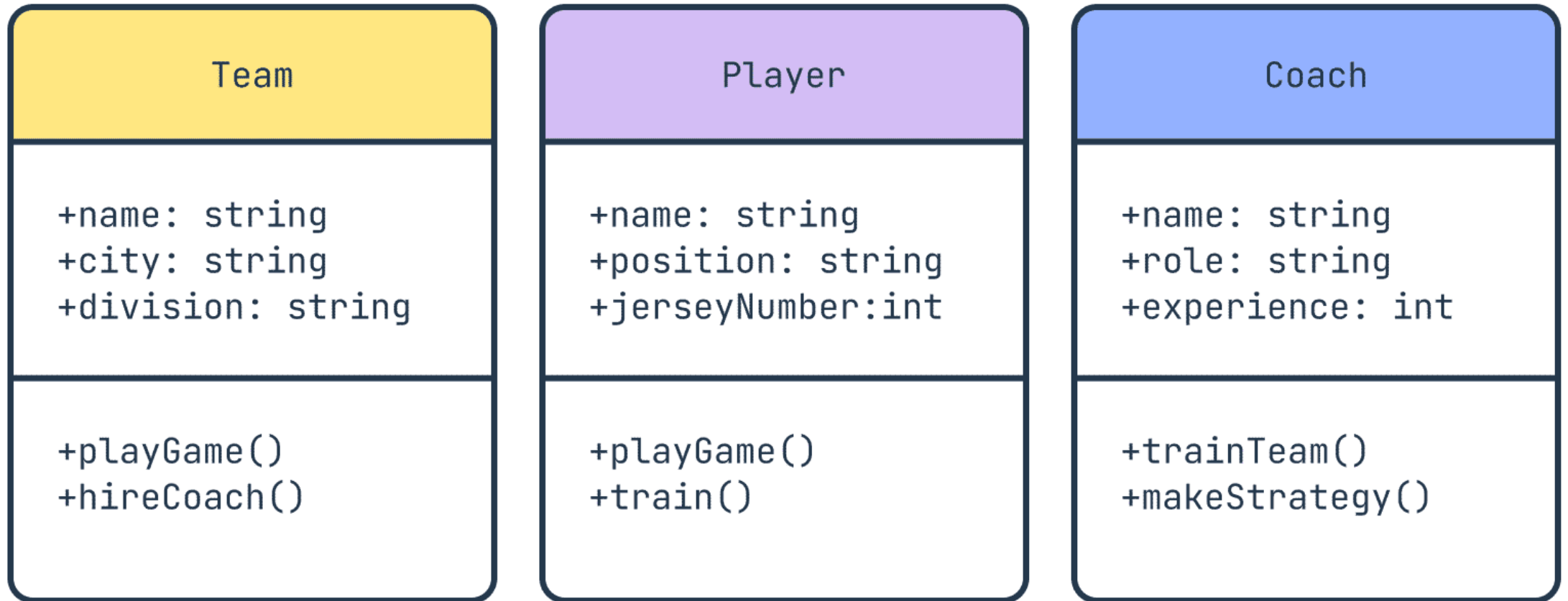
- **Equipo** → jugar un partido, contratar entrenador.
- **Jugador** → jugar un partido, entrenar.
- **Entrenador** → entrenar equipo, confeccionar estrategia.



5) Definir las relaciones



6) Confeccionar el diagrama UML





Tarea: diagramas de clase UML

Ejercicio 1

Una empresa de alquiler de vehículos necesita un sistema para gestionar su flota. Existen distintos tipos de vehículos, como autos y motos, los cuales comparten ciertas características como marca, modelo y año de fabricación, pero de los autos también se registran la cantidad de puertas y de las motos el tipo de manubrio. Cada vehículo puede ser alquilado por un cliente, del cual se registran su nombre, apellido, DNI y un número de licencia de conducir. Un cliente puede alquilar varios vehículos (entre dos fechas puntuales), pero cada vehículo solo puede estar alquilado a un cliente a la vez.

El sistema también debe arrojar cierta información útil: determinar si el vehículo es antiguo (10 o más años); ver los detalles completos de cada moto y auto; ver el historial de alquiler de cada cliente de la empresa; y averiguar cuanto tiempo falta para que finalice un alquiler actual.

Tarea: diagramas de clase UML

Ejercicio 2

Una tienda en línea maneja pedidos realizados por clientes. Cada pedido puede contener múltiples productos, y un cliente puede realizar varios pedidos. Un producto tiene un código único, un nombre, un precio y una cantidad en stock. Cada pedido tiene una fecha y un estado (pendiente, enviado o entregado); este estado se puede modificar en cualquier momento, así como también obtener el precio total. Los productos están agregados en un pedido, lo que significa que un producto puede existir fuera del pedido y ser parte de múltiples pedidos. Los clientes se registran en el sistema con nombre, apellido e email, y además de realizar pedidos pueden ver el historial de sus pedidos previos.

Otras actividades propias del sistema incluyen actualizar el stock de cada producto cuando se modifiquen sus existencias y obtener un listado con la información completa de un producto

Ejercicio 3

Una universidad necesita un sistema para gestionar su biblioteca digital. En ella, los usuarios pueden acceder a una variedad de documentos digitales, como libros electrónicos, tesis y artículos científicos. Cada tipo de documento tiene atributos comunes como título, fecha de publicación y número de páginas, pero también tienen características propias: los libros electrónicos tienen un ISBN y un autor principal; las tesis incluyen el nombre del director de tesis y el grado académico asociado; y los artículos científicos indican la revista en la que fueron publicados y su número de volumen.

Cada usuario tiene un nombre completo, una dirección de correo electrónico institucional y un identificador único generado automáticamente a partir de su nombre y número de matrícula. Los usuarios pueden descargar documentos, y cada descarga registra la fecha y hora, el tipo de dispositivo usado y la dirección IP desde la que se accedió.

Además, los documentos se organizan por categorías temáticas, las cuales tienen un nombre, un código alfanumérico y una breve descripción. Un documento puede pertenecer a varias categorías, y cada categoría puede agrupar muchos documentos.